

Politechnika Warszawska

WYDZIAŁ ELEKTRONIKI
I TECHNIK INFORMACYJNYCH



Instytut Telekomunikacji

Praca dyplomowa magisterska

na kierunku Telekomunikacja
w specjalności Teleinformatyka i Cyberbezpieczeństwo

Autonomiczne pętle sterowania cyklem życia usług w środowisku
Kubernetes

Jakub Kokoszka

Numer albumu 304154

promotor
dr. inż. Dariusz Bursztynowski

WARSZAWA 2026

Słowa kluczowe: XXX, XXX, XXX

Autonomous control loops for service lifecycle management in Kubernetes environment

Abstract. As any dedicated reader can clearly see, the Ideal of practical reason is a representation of, as far as I know, the things in themselves; as I have shown elsewhere, the phenomena should only be used as a canon for our understanding. The paralogisms of practical reason are what first give rise to the architectonic of practical reason. As will easily be shown in the next section, reason would thereby be made to contradict, in view of these considerations, the Ideal of practical reason, yet the manifold depends on the phenomena. Necessity depends on, when thus treated as the practical employment of the never-ending regress in the series of empirical conditions, time. Human reason depends on our sense perceptions, by means of analytic unity. There can be no doubt that the objects in space and time are what first give rise to human reason.

Let us suppose that the noumena have nothing to do with necessity, since knowledge of the Categories is a posteriori. Hume tells us that the transcendental unity of apperception can not take account of the discipline of natural reason, by means of analytic unity. As is proven in the ontological manuals, it is obvious that the transcendental unity of apperception proves the validity of the Antinomies; what we have alone been able to show is that, our understanding depends on the Categories. It remains a mystery why the Ideal stands in need of reason. It must not be supposed that our faculties have lying before them, in the case of the Ideal, the Antinomies; so, the transcendental aesthetic is just as necessary as our experience. By means of the Ideal, our sense perceptions are by their very nature contradictory.

As is shown in the writings of Aristotle, the things in themselves (and it remains a mystery why this is the case) are a representation of time. Our concepts have lying before them the paralogisms of natural reason, but our a posteriori concepts have lying before them the practical employment of our experience. Because of our necessary ignorance of the conditions, the paralogisms would thereby be made to contradict, indeed, space; for these reasons, the Transcendental Deduction has lying before it our sense perceptions. (Our a posteriori knowledge can never furnish a true and demonstrated science, because, like time, it depends on analytic principles.) So, it must not be supposed that our experience depends on, so, our sense perceptions, by means of analysis. Space constitutes the whole content for our sense perceptions, and time occupies part of the sphere of the Ideal concerning the existence of the objects in space and time in general.

Keywords: XXX, XXX, XXX



.....
miejscowość i data

.....
imię i nazwisko studenta

.....
numer albumu

.....
kierunek studiów

OŚWIADCZENIE

Świadomy/-a odpowiedzialności karnej za składanie fałszywych zeznań oświadczam, że niniejsza praca dyplomowa została napisana przeze mnie samodzielnie, pod opieką kierującego pracą dyplomową.

Jednocześnie oświadczam, że:

- niniejsza praca dyplomowa nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (Dz.U. z 2006 r. Nr 90, poz. 631 z późn. zm.) oraz dóbr osobistych chronionych prawem cywilnym,
- niniejsza praca dyplomowa nie zawiera danych i informacji, które uzyskałem/-am w sposób niedozwolony,
- niniejsza praca dyplomowa nie była wcześniej podstawą żadnej innej urzędowej procedury związanej z nadawaniem dyplomów lub tytułów zawodowych,
- wszystkie informacje umieszczone w niniejszej pracy, uzyskane ze źródeł pisanych i elektronicznych, zostały udokumentowane w wykazie literatury odpowiednimi odnośnikami,
- znam regulacje prawne Politechniki Warszawskiej w sprawie zarządzania prawami autorskimi i prawami pokrewnymi, prawami własności przemysłowej oraz zasadami komercjalizacji.

Oświadczam, że treść pracy dyplomowej w wersji drukowanej, treść pracy dyplomowej zawartej na nośniku elektronicznym (płyce kompaktowej) oraz treść pracy dyplomowej w module APD systemu USOS są identyczne.

.....
czytelny podpis studenta

Spis treści

1. Wstęp

1.1. Wprowadzenie do problematyki

Dynamiczny rozwój technologii chmurowych oraz wzrost złożoności systemów rozproszonych stanowią wyzwanie dla efektywnego zarządzania zasobami i zapewnienia niezawodności usług. W dobie mikroserwisów i architektur opartych na kontenerach, platformy takie jak Kubernetes stały się de facto standardem w orkiestracji aplikacji, umożliwiając deklaratywne zarządzanie infrastrukturą. Niemniej jednak, samo wdrożenie aplikacji w Kubernetesie nie gwarantuje optymalnego wykorzystania zasobów ani automatycznego dostosowywania się do zmieniających się warunków obciążenia. Konieczność manualnego skalowania lub konfiguracji parametrów alokacji zasobów prowadzi do nieefektywności operacyjnej, zwiększonych kosztów oraz potencjalnych przestojów, wynikających z niedoszacowania lub przeszacowania potrzeb zasobowych. Współczesne systemy wymagają mechanizmów, które autonomicznie adaptują się do dynamicznych zmian w środowisku, minimalizując interwencję człowieka. W tym kontekście, autonomiczne pętle sterowania cyklem życia usług, zdolne do monitorowania, analizowania, planowania i wykonywania akcji bez nadzoru, stają się kluczowe dla osiągnięcia wysokiej dostępności, wydajności i ekonomiczności w środowiskach kontenerowych, co potwierdzają liczne badania w dziedzinie samoadaptujących się systemów. Poszukiwanie innowacyjnych rozwiązań, które efektywnie zarządzają skalowaniem i alokacją zasobów, stanowi zatem istotny obszar badań, mający bezpośrednie przełożenie na stabilność i efektywność nowoczesnych aplikacji chmurowych.

1.2. Kontekst i motywacja wyboru tematu

Współczesne środowiska chmurowe charakteryzują się rosnącą złożonością oraz dynamicznie zmieniającym się obciążeniem aplikacji. Przedsiębiorstwa coraz częściej migrują swoje systemy do architektur mikroserwisowych opartych na kontenerach, zarządzanych przez platformy orkiestracyjne takie jak Kubernetes. Umożliwia to znaczące zwiększenie elastyczności, skalowalności oraz efektywności operacyjnej. Jednocześnie jednak zarządzanie cyklem życia usług w tak dynamicznych ekosystemach stanowi istotne wyzwanie. Tradycyjne, statyczne podejścia do przydzielania zasobów lub ręczne procesy skalowania nie są w stanie efektywnie reagować na zmienne warunki obciążenia, prowadząc do nieoptymalnego wykorzystania zasobów lub spadków wydajności w momentach szczytowego ruchu.

W odpowiedzi na te wyzwania, w środowisku Kubernetes rozwijane są różne mechanizmy automatycznego skalowania, takie jak Horizontal Pod Autoscaler (HPA), Vertical Pod Autoscaler (VPA) czy Kubernetes Event-driven Autoscaling (KEDA). Każdy z nich posiada jednak swoje ograniczenia — HPA skupia się głównie na skalowaniu replik, ignorując możliwości optymalizacji zasobów pojedynczego kontenera, podczas gdy VPA dokonuje

zmian w przydziale zasobów, lecz nie radzi sobie dobrze w sytuacjach nagłych skoków obciążenia. W rezultacie żadne z tych rozwiązań nie zapewnia optymalnego balansu między stabilnością, wydajnością a kosztami w zróżnicowanych warunkach pracy aplikacji.

Motywacją do podjęcia tematu pracy magisterskiej jest potrzeba opracowania autonomicznego mechanizmu, który byłby w stanie samodzielnie i dynamicznie decydować o wyborze właściwej strategii skalowania w zależności

1.3. Cel pracy i problemy badawcze

Głównym celem niniejszej pracy magisterskiej jest opracowanie, implementacja oraz ewaluacja autonomicznego operatora Kubernetes, zdolnego do dynamicznego wyboru między pionowym VPA a poziomym HPA skalowaniem na podstawie bieżących warunków obciążenia oraz zdefiniowanej polityki. Realizacja tego celu ma na celu wykazanie, że zastosowanie pętli sterującej opartej na modelu Monitor, Analize, Plan, Execute, Knowledge (MAPE-K) może prowadzić do zwiększenia efektywności wykorzystania zasobów oraz stabilności działania usług w środowiskach chmurowych.

Praca zakłada analizę istniejących mechanizmów automatycznego skalowania w ekosystemie Kubernetes, takich jak HPA oraz VPA w celu identyfikacji ich ograniczeń i obszarów wymagających ulepszenia. Na tej podstawie zaprojektowane zostanie autorskie rozwiązanie w postaci operatora, który – korzystając z dedykowanego zasobu Custom Resource Definition (CRD) – będzie decydował o aktywacji odpowiedniego mechanizmu skalowania w sposób autonomiczny, zależny od bieżących danych metrycznych i ustalonej polityki działania. Celem końcowym jest porównanie skuteczności zaproponowanego podejścia z istniejącymi metodami w kontekście wydajności, stabilności oraz efektywności kosztowej.

W kontekście powyższych założeń, niniejsza praca stawia następujące **problemy badawcze**:

1. Jakie są ograniczenia istniejących mechanizmów skalowania w Kubernetesie (HPA, VPA) w kontekście efektywności wykorzystania zasobów oraz zdolności adaptacyjnych do dynamicznie zmieniających się obciążeń?
2. W jaki sposób autonomiczny operator, wykorzystujący model MAPE-K, może podejmować decyzje o wyborze odpowiedniego typu skalowania (HPA lub VPA), aby poprawić stabilność i efektywność pracy usług?
3. Czy zaproponowane rozwiązanie pozwala uzyskać lepsze rezultaty w zakresie **wydajności, stabilności i kosztów** w porównaniu do istniejących mechanizmów autoskalowania?
4. Jakie metryki i kryteria oceny należy przyjąć, aby w sposób obiektywny zmierzyć skuteczność działania autonomicznego operatora w środowisku zbliżonym do produkcyjnego?

1.4. Hipotezy badawcze

W kontekście zdefiniowanej tezy głównej oraz szczegółowych celów i problemów badawczych, sformułowano następujące hipotezy, które zostaną zweryfikowane w trakcie przeprowadzonych eksperymentów i analiz:

1. **Istniejące mechanizmy skalowania w Kubernetesie**, takie jak HPA i VPA, pomimo swojej skuteczności w określonych scenariuszach, nie zapewniają optymalnego wykorzystania zasobów w warunkach dynamicznie zmieniającego się obciążenia. Ich ograniczona zdolność adaptacyjna może prowadzić do przeskalowania lub niedoskalowania aplikacji, a w konsekwencji do nieefektywności kosztowej lub pogorszenia jakości usług.
2. **Autonomiczny operator Kubernetes**, zaprojektowany i zaimplementowany w ramach niniejszej pracy, wykorzystujący model pętli sterującej MAPE-K do dynamicznego wyboru między HPA a VPA, zapewni większą efektywność wykorzystania zasobów oraz wyższą stabilność działania usług w porównaniu do stosowania pojedynczych mechanizmów skalowania.
3. **Zastosowanie modelu MAPE-K** w procesie zarządzania cyklem życia usług pozwoli na zwiększenie autonomii systemu i redukcję konieczności interwencji manualnej, co przyczyni się do ograniczenia kosztów operacyjnych oraz poprawy dostępności i niezawodności aplikacji działających w środowisku chmurowym.

2. Podstawy Teoretyczne i Przegląd Literatury

Niniejszy rozdział ma na celu przedstawienie kluczowych koncepcji teoretycznych oraz przegląd istniejącej literatury naukowej, niezbędnych do pełnego zrozumienia problematyki autonomicznego sterowania cyklem życia usług w środowisku Kubernetes. Rozpoczyna się od omówienia podstaw architektury Kubernetesa, jego komponentów oraz mechanizmów rozszerzeń, takich jak CRD, które stanowią platformę dla implementowanych rozwiązań. Następnie, w kontekście rosnącej złożoności systemów rozproszonych, przedstawione zostaną fundamentalne zasady automatyzacji zarządzania zasobami, ze szczególnym uwzględnieniem koncepcji pętli sterowania oraz systemów samoadaptujących się.

Kluczową częścią rozdziału jest szczegółowy przegląd i analiza istniejących mechanizmów skalowania w Kubernetesie, takich jak VPA, HPA oraz KEDA, z uwzględnieniem ich zasad działania, ograniczeń i typowych zastosowań. Rozwiązania te zostaną przeanalizowane w kontekście automatycznego zarządzania zasobami obliczeniowymi oraz reakcji systemu na zmienne obciążenie.

Rozdział zostanie uzupełniony o wprowadzenie do modelu MAPE-K (ang. *Monitor, Analyze, Plan, Execute, Knowledge*), który stanowi uznany paradygmat projektowania autonomicznych systemów zarządzania i będzie podstawą do opracowania własnego rozwiązania w dalszych częściach pracy. Zebrana w tym rozdziale wiedza teoretyczna stworzy solidne ramy dla przeprowadzenia badań empirycznych oraz analizy uzyskanych wyników.

2.1. Architektura i koncepcje Kubernetesa

Kubernetes, będący otwartym systemem do automatyzacji wdrażania, skalowania oraz zarządzania aplikacjami kontenerowymi, stanowi obecnie dominującą platformę orkiestracyjną w ekosystemach chmurowych. Jego architektura opiera się na modelu *master-node*, w którym **kłaster Kubernetesa** składa się z Control Plane (CP), zazwyczaj uruchamianej na węzłach typu *master*, oraz z wielu Worker Node (WN), na których działają kontenery z aplikacjami.

2.1.1. Podstawowe komponenty: control plane, nodes, pods, deployments, services

CP obejmuje kluczowe komponenty systemowe, takie jak **kube-apiserver**, który udostępnia interfejs API Kubernetesa i stanowi centralny punkt komunikacji w klastrze; **kube-scheduler**, odpowiedzialny za przydzielanie nowo tworzonych Pod (Pod) do odpowiednich węzłów; **kube-controller-manager**, zarządzający zestawem kontrolerów realizujących logikę utrzymania pożądanego stanu systemu (m.in. kontroler replikacji czy kontroler punktów końcowych); oraz **etcd**, będący rozproszoną bazą danych typu klucz-wartość, przechowującą aktualny stan klastra.

Każdy WN zawiera komponent **kubelet**, pełniący rolę agenta komunikującego się z płaszczyzną sterowania oraz zarządzającego cyklem życia podów i kontenerów na danym węźle, a także **kube-proxy**, który odpowiada za implementację reguł sieciowych dla Service (Service), umożliwiając komunikację pomiędzy podami oraz dostęp do usług zewnętrznych.

2.1.2. Mechanizmy rozszerzeń: CRDs i operatory

Kubernetes operuje na zestawie abstrakcji, takich jak Pod, stanowiące najmniejszą jednostkę wdrożeniową i grupujące jeden lub więcej kontenerów; Deployment (Deployment), zapewniające deklaratywne zarządzanie stanem aplikacji, w tym automatyzację aktualizacji oraz wycofywania zmian; oraz Service, będące abstrakcjami sieciowymi definiującymi logiczny zbiór podów oraz politykę dostępu do nich.

Elastyczność platformy jest dodatkowo rozszerzana poprzez mechanizmy takie jak CRD, które umożliwiają definiowanie własnych obiektów API i integrację niestandardowych zasobów z natywnym modelem sterowania Kubernetesa. Na ich bazie budowane są Kubernetes Operator (Operator), czyli komponenty oprogramowania enkapsulujące wiedzę domenową i automatyzujące zarządzanie złożonymi aplikacjami oraz ich cyklem życia w klastrze. Zrozumienie tych fundamentalnych elementów stanowi podstawę do analizy mechanizmów autonomicznego sterowania, które wykorzystują i rozszerzają natywne możliwości platformy Kubernetes.

2.2. Automatyzacja zarządzania zasobami w chmurze

Automatyzacja zarządzania zasobami w chmurze obliczeniowej jest fundamentalnym elementem wydajnych, niezawodnych i skalowalnych systemów informatycznych. Opiera się na wdrażaniu zasad automatyki i inżynierii sterowania, umożliwiając inteligentne, często autonomiczne zarządzanie infrastrukturą IT. W tym kontekście kluczowym pojęciem pozostaje pętla sterowania (control loop), będąca podstawą systemów samonaprawiających się oraz adaptacyjnych rozwiązań chmurowych.

2.2.1. Pojęcie pętli sterowania (Control Loops) w systemach rozproszonych

Pętla sterowania to podstawowy mechanizm automatyki, polegający na nieustannym monitorowaniu parametrów systemu, ich analizie oraz podejmowaniu działań korygujących, jeśli wykryta zostanie odchyłka od zadanych wartości. Pętle te mogą być zarówno otwarte, jak i zamknięte („feedback loops”), jednak w praktyce zarządzania chmurą dominują warianty z informacją zwrotną.

Historycznie, idea pętli sterowania wywodzi się z XVIII wieku, gdy James Watt opracował regulator odśrodkowy dla maszyn parowych, umożliwiając automatyczne utrzymywanie zadanej prędkości obrotowej silnika poprzez mechaniczne sprzężenie zwrotne.

W architekturach rozproszonych, jak chmura obliczeniowa czy systemy DCS (Distributed Control Systems), pętle sterowania realizowane są na wielu poziomach – od lokalnych

kontrolerów odpowiedzialnych za pojedyncze węzły, po nadzorczy poziom systemowy. Przykłady obejmują:

- *Load Balancing Feedback Loops*: dynamiczne równoważenie obciążenia na podstawie bieżących informacji o stanie zasobów;
- *Scaling Feedback Loops*: automatyczna zmiana liczby aktywnych instancji usług zależnie od aktualnych potrzeb;
- *Health Monitoring Feedback Loops*: wykrywanie oraz samoczynna reakcja na awarie lub degradację wydajności.

Pętle te zapewniają adaptacyjność oraz samonaprawialność systemów, pozwalając na minimalizowanie wpływu awarii, optymalizację zużycia zasobów oraz ciągłość działania usług.

2.2.2. Autonomiczne systemy i samoadaptacja

Ewolucja automatyki i inżynierii sterowania doprowadziła do powstania **autonomicznych systemów**, które nie tylko wykonują zaprogramowane wcześniej zadania, lecz potrafią adaptować swoje zachowanie do zmieniających się warunków środowiska i własnego stanu. Zasadniczym mechanizmem jest tu samoadaptacja (self-adaptation), polegająca na dynamicznej zmianie parametrów lub sposobu działania systemu na podstawie analizy bieżących danych.

W systemach autonomicznych typowe są zaawansowane architektury warstwowe, gdzie warstwa zarządzająca (managing subsystem) monitoruje „kondycję” systemu oraz środowiska i na tej podstawie podejmuje decyzje dotyczące rekonfiguracji czy optymalizacji:

- Przykładem mogą być pojazdy autonomiczne (np. podwodne roboty inspekcyjne), których sterownik może dynamicznie przełączać tryby działania – np. powrót do stacji ładowania przy niskim poziomie energii, czy wybór mniej energochłonnego algorytmu wykrywania przeszkód w przypadku rosnących trudności środowiskowych ;
- W chmurze obliczeniowej takie mechanizmy wykorzystywane są do dynamicznej alokacji zasobów, rekonfiguracji po awariach, czy inteligentnego dostrajania parametrów usług w odpowiedzi na nieprzewidywalne zmiany obciążenia .

Badania naukowe wskazują, że skuteczność samoadaptacji i autonomiczności w systemach rozproszonych w dużej mierze zależy od efektywności sprzężenia zwrotnego, precyzyjnego modelowania stanu systemu oraz algorytmów decyzyjnych uwzględniających zarówno aspekty wydajnościowe, jak i bezpieczeństwo czy energooszczędność.

Rozwój autonomicznych i samoadaptacyjnych systemów w chmurze znajduje potwierdzenie w licznych badaniach, zarówno w obszarze zastosowań przemysłowych, jak i rozwoju koncepcji architektonicznych oraz formalnych metod modelowania, analiz i wалиdacji poprawności działania tych systemów. To właśnie te interdyscyplinarne innowacje –

czerpiące z tradycyjnej automatyki, inżynierii oprogramowania, robotyki czy informatyki stosowanej – umożliwiają konsekwentną automatyzację i dalszy rozwój chmury obliczeniowej.

2.3. Przegląd Istniejących Mechanizmów Skalowania i Zarządzania

W kontekście dynamicznego środowiska Kubernetesa, efektywne zarządzanie zasobami i skalowanie aplikacji jest kluczowe dla zapewnienia wydajności, stabilności oraz optymalizacji kosztów. Kubernetes oferuje natywne mechanizmy automatycznego skalowania, które pozwalają na dostosowanie liczby instancji aplikacji (skalowanie horyzontalne) oraz zasobów przydzielonych pojedynczym instancjom (skalowanie wertykalne) w odpowiedzi na zmieniające się zapotrzebowanie. Ponadto, ekosystem Kubernetesa jest stale rozwijany o rozwiązania zewnętrzne, które rozszerzają te możliwości, umożliwiając skalowanie oparte na zdarzeniach pochodzących z różnych źródeł. Poniższe podsekcje przedstawiają szczegółową analizę kluczowych mechanizmów używanych do autonomicznego skalowania w Kubernetesie.

2.3.1. Vertical Pod Autoscaler (VPA): zasada działania, ograniczenia, zastosowania

Vertical Pod Autoscaler (VPA) jest mechanizmem Kubernetesa, który automatycznie dostosowuje żądania (requests) i limity (limits) zasobów CPU i pamięci dla pojedynczych Podów. Jego podstawowa zasada działania opiera się na ciągłym monitorowaniu rzeczywistego zużycia zasobów przez kontenery w Podzie. Na podstawie historycznych danych o wykorzystaniu, VPA rekomenduje lub automatycznie aplikuje optymalne wartości dla żądań i limitów zasobów, dążąc do minimalizacji marnotrawstwa zasobów przy jednoczesnym zapewnieniu stabilnej pracy aplikacji. VPA składa się z trzech głównych komponentów: **Recommender**, który analizuje metryki zużycia zasobów i proponuje optymalne wartości; **Updater**, który faktycznie modyfikuje specyfikację Podów lub, w trybie automatycznym, reinicjuje Pody z nowymi ustawieniami zasobów; oraz **Admission Controller**, który przechwytuje żądania tworzenia Podów i nadpisuje ich zasoby zgodnie z rekomendacjami VPA.

Mimo swoich zalet, VPA posiada pewne ograniczenia. Jednym z głównych jest fakt, że automatyczne zastosowanie zmian zasobów zazwyczaj wymaga zrestartowania Poda, co może prowadzić do krótkotrwałych przerw w dostępności usługi. Ponadto, VPA najlepiej sprawdza się w przypadku aplikacji o stosunkowo przewidywalnym wzorcu zużycia zasobów, natomiast w aplikacjach o bardzo dynamicznych i nieregularnych skokach obciążenia może nie reagować wystarczająco szybko. Co więcej, VPA domyślnie rekomenduje wartości dla wszystkich kontenerów w Podzie, co może być nieoptymalne dla Podów z wieloma kontenerami o zróżnicowanych profilach zużycia. Zastosowania VPA obejmują głównie aplikacje, dla których kluczowa jest optymalizacja kosztów i wykorzystania pojedynczych instancji, takie jak bazy danych, systemy cache czy aplikacje monolityczne o

stabilnym profilu obciążenia, gdzie precyzyjne dopasowanie zasobów jest ważniejsze niż natychmiastowa reakcja na nagłe piki.

2.3.2. Horizontal Pod Autoscaler (HPA): zasada działania, metryki, skalowanie oparte na obciążeniu

Horizontal Pod Autoscaler (HPA) to mechanizm Kubernetesa odpowiedzialny za automatyczne skalowanie liczby replik Podów w zależności od obserwowanego obciążenia. Jego główna zasada działania polega na monitorowaniu wybranych metryk (najczęściej zużycia CPU i pamięci, ale także metryk niestandardowych lub zewnętrznych) i porównywaniu ich z zdefiniowanymi progami. Jeśli wartość metryki przekroczy określony próg, HPA zwiększa liczbę replik Podów; jeśli spadnie poniżej progu, zmniejsza ich liczbę, dążąc do utrzymania średniego zużycia zasobów na poziomie zbliżonym do wartości docelowej. HPA działa jako kontroler w płaszczyźnie sterowania Kubernetesa, regularnie sprawdzając metryki z Metrics API (lub innych źródeł) i aktualizując pole 'replicas' w obiektach takich jak Deployment, ReplicaSet czy StatefulSet.

Kluczową zaletą HPA jest możliwość skalowania opartego na obciążeniu, co pozwala na efektywne zarządzanie dostępnością i wydajnością aplikacji w zmiennym środowisku. Metryki używane przez HPA mogą pochodzić z różnych źródeł:

- **Metryki zasobów (Resource Metrics):** najczęściej używane są metryki dotyczące zużycia CPU (wyrażone jako procent żądanej wartości CPU) oraz pamięci. Są one dostarczane przez Metrics Server, który zbiera dane z kubeletów.
- **Metryki niestandardowe (Custom Metrics):** umożliwiają skalowanie w oparciu o metryki specyficzne dla aplikacji (np. liczba żądań na sekundę, długość kolejki wiadomości), dostarczane przez adaptery Custom Metrics API.
- **Metryki zewnętrzne (External Metrics):** pozwalają na skalowanie w oparciu o metryki pochodzące spoza klastra Kubernetes (np. długość kolejki w systemie messagingowym takim jak Kafka czy RabbitMQ), dostarczane przez adaptery External Metrics API.

HPA jest szczególnie efektywny dla aplikacji typu stateless, które mogą łatwo być skalowane horyzontalnie poprzez dodawanie lub usuwanie replik. Jego ograniczenia wynikają z opóźnień w reakcji na gwałtowne zmiany obciążenia (ze względu na konieczność uruchomienia nowych Podów) oraz z potrzeby precyzyjnego ustawienia progów metryk, co może być wyzwaniem w przypadku nieprzewidywalnych wzorców ruchu. Mimo to, HPA jest podstawowym narzędziem do utrzymania wysokiej dostępności i reagowania na piki obciążenia w większości wdrożeń Kubernetesa.

2.3.3. Kubernetes Event-driven Autoscaling (KEDA): integracja z brokerami zdarzeń, scenariusze użycia

Kubernetes Event-driven Autoscaling (KEDA) to elastyczne i rozszerzalne rozwiązanie typu open-source, które uzupełnia i rozszerza funkcjonalności Horizontal Pod Autoscaler

(HPA), umożliwiając skalowanie aplikacji w Kubernetesie w oparciu o liczbę zdarzeń w kolejkach, strumieniach czy innych systemach zewnętrznych. KEDA działa jako operator Kubernetesa, który dynamicznie tworzy obiekty HPA na podstawie zdefiniowanych **Scalerów**, a następnie usuwa je, gdy nie są już potrzebne. To pozwala na zero-skalowanie, czyli redukcję liczby replik do zera, gdy nie ma żadnych zdarzeń do przetworzenia, co znacząco optymalizuje koszty.

Kluczową cechą KEDA jest jego głęboka integracja z brokerami zdarzeń i zewnętrznymi systemami. KEDA oferuje szeroką gamę wbudowanych Scalerów dla popularnych technologii, takich jak Apache Kafka, RabbitMQ, Azure Service Bus, AWS SQS, Redis, Prometheus czy baza danych PostgreSQL. Dzięki temu aplikacje, które przetwarzają zdarzenia asynchronicznie, mogą być skalowane w sposób precyzyjny i efektywny, reagując bezpośrednio na bieżące zapotrzebowanie.

Typowe scenariusze użycia KEDA obejmują:

- **Aplikacje oparte na kolejkach wiadomości:** skalowanie workerów konsumujących wiadomości z kolejek, gdzie liczba replik jest wprost proporcjonalna do liczby oczekujących wiadomości.
- **Funkcje serverless (Function-as-a-Service):** umożliwienie uruchamiania funkcji tylko wtedy, gdy pojawią się zdarzenia, minimalizując zużycie zasobów w czasie bezczynności.
- **Przetwarzanie strumieni danych:** adaptacyjne skalowanie aplikacji przetwarzających strumień danych w czasie rzeczywistym, reagując na zmieniającą się przepustowość strumienia.
- **Integracja z bazami danych i systemami pamięci podręcznej:** skalowanie w oparciu o liczbę zapytań lub obciążenie tych systemów.

KEDA jest szczególnie cenne w architekturach event-driven, gdzie HPA oparte na CPU lub pamięci byłoby niewystarczające. Jego elastyczność i możliwość łatwego dodawania nowych Scalerów sprawiają, że jest to potężne narzędzie do budowania wysoce adaptacyjnych i ekonomicznych systemów w Kubernetesie.

2.4. Model MAPE-K w Systemach Samozarządzających

W obliczu rosnącej złożoności i dynamiki współczesnych systemów informatycznych, zwłaszcza tych rozproszonych i opartych na chmurze, tradycyjne podejścia do zarządzania stają się niewystarczające. Konieczność minimalizacji interwencji ludzkiej, zwiększenia odporności na awarie oraz optymalizacji wydajności i kosztów doprowadziła do rozwoju koncepcji systemów samozarządzających. Kluczowym paradygmatem w tym obszarze jest model MAPE-K, stanowiący ramy dla projektowania i implementacji autonomicznych mechanizmów kontrolnych. Model ten dostarcza ustrukturyzowane podejście do budowania systemów zdolnych do samokonfiguracji, samooptymalizacji, samonaprawy i samoochrony, w dużej mierze eliminując potrzebę ciągłego nadzoru człowieka.

2.4.1. Monitor, Analize, Plan, Execute, Knowledge

Model MAPE-K składa się z czterech głównych komponentów funkcjonalnych tworzących pętlę sprzężenia zwrotnego, wspieraną przez wspólną bazę wiedzy. Każdy z tych elementów odgrywa kluczową rolę w procesie autonomicznego zarządzania:

- **Monitor (Monitoruj):** Ten komponent jest odpowiedzialny za zbieranie danych o systemie i jego środowisku. Obejmuje to gromadzenie metryk wydajności (np. zużycie CPU, pamięci, opóźnienia, przepustowość), informacji o stanie komponentów, logów błędów oraz wszelkich innych danych kontekstowych, które są istotne dla podejmowania decyzji. Skuteczne monitorowanie wymaga odpowiedniej instrumentacji systemu i zdolności do agregacji danych z wielu źródeł w czasie rzeczywistym.
- **Analize (Analizuj):** Moduł analizy przetwarza zebrane dane, identyfikując wzorce, trendy oraz anomalie. Jego zadaniem jest diagnozowanie problemów, przewidywanie przyszłych stanów systemu oraz ocenianie, czy obecny stan odpowiada pożądanym celom. Analiza może obejmować zastosowanie statystyk, algorytmów uczenia maszynowego do wykrywania korelacji, predykcji obciążenia czy identyfikacji pierwotnych przyczyn problemów. W tym etapie system rozumie "co się dzieje i dlaczego".
- **Plan (Planuj):** Na podstawie wyników analizy, komponent planowania generuje sekwencję akcji, które mają na celu dostosowanie systemu do pożądanego stanu lub rozwiązanie zdiagnozowanego problemu. Planowanie obejmuje wybór strategii adaptacji, optymalizację zasobów, definiowanie zmian konfiguracji lub modyfikację polityk. Ten etap odpowiada na pytanie "co należy zrobić".
- **Execute (Wykonaj):** Moduł wykonawczy jest odpowiedzialny za implementację planu. Przekształca on abstrakcyjne akcje zdefiniowane w fazie planowania na konkretne operacje w systemie, takie jak skalowanie instancji, zmiana konfiguracji, restartowanie komponentów czy relokacja obciążenia. Wykonanie musi być odporne na błędy i zapewniać spójność systemu.
- **Knowledge (Wiedza):** Komponent wiedzy stanowi centralne repozytorium informacji, które jest współdzielone przez wszystkie pozostałe moduły. Zawiera on cele systemu (np. polityki QoS, SLA), modele środowiska i aplikacji, historyczne dane o zachowaniu systemu, a także zasady i algorytmy wykorzystywane w analizie i planowaniu. Baza wiedzy jest dynamiczna i może być aktualizowana na podstawie doświadczeń zebranych podczas działania pętli.

2.4.2. Aplikacja modelu MAPE-K w zarządzaniu usługami

Model MAPE-K jest powszechnie stosowany jako architektura referencyjna dla systemów samozarządzających w różnorodnych domenach, w tym w zarządzaniu usługami w chmurze obliczeniowej. Jego modularność i jasno zdefiniowane funkcje sprawiają, że ide-

alnie nadaje się do projektowania autonomicznych kontrolerów, które mogą dynamicznie adaptować usługi do zmieniających się warunków.

W kontekście zarządzania usługami, aplikacja modelu MAPE-K może przybrać następujące formy:

- **Monitorowanie Usług:** obejmuje zbieranie metryk dotyczących dostępności, wydajności (np. czasy odpowiedzi, przepustowość), zużycia zasobów (CPU, pamięć, sieć, dysk) przez poszczególne usługi i ich komponenty. Monitorowane są również metryki biznesowe i zdarzenia pochodzące z zewnętrznych systemów.
- **Analiza Zachowania Usług:** polega na analizie zebranych danych w celu wykrycia anomalii, przewidywania przyszłego obciążenia, identyfikacji wąskich gardeł lub określenia, czy usługi spełniają zdefiniowane poziomy SLA (Service Level Agreement). Analiza może obejmować agregację danych, tworzenie prognoz czy uruchamianie algorytmów detekcji wzorców.
- **Planowanie Adaptacji Usług:** na podstawie wyników analizy, komponent planowania podejmuje decyzje o koniecznych zmianach. Może to być decyzja o skalowaniu horyzontalnym (dodanie/usunięcie instancji), skalowaniu wertykalnym (zmiana zasobów Podu), relokacji usług, optymalizacji konfiguracji czy wdrożeniu poprawek. Plan jest optymalizowany pod kątem wielu kryteriów, takich jak koszt, wydajność i odporność.
- **Wykonanie Akcji Adaptacyjnych:** moduł wykonawczy realizuje zaplanowane zmiany poprzez interakcję z platformą orkiestracji (np. API Kubernetesa). Obejmuje to uruchamianie/zatrzymywanie Podów, modyfikowanie konfiguracji, aktualizowanie reguł sieciowych czy dostosowywanie parametrów bazy danych. Ważne jest, aby wykonanie było atomowe i bezpieczne, aby nie wprowadzać niestabilności.
- **Baza Wiedzy dla Usług:** Przechowuje informacje o politykach zarządzania usługami, topologii sieci, zależnościach między mikroservisami, historycznych profilach obciążenia, predefiniowanych strategiach skalowania, a także modelach predykcyjnych. Wiedza ta jest wykorzystywana na każdym etapie pętli, ewoluując wraz z doświadczeniem systemu.

Model MAPE-K zapewnia elastyczne ramy do budowania autonomicznych systemów zarządzania usługami, które są w stanie efektywnie reagować na dynamiczne warunki operacyjne, optymalizować wykorzystanie zasobów i zapewniać wysoką jakość usług bez ciągłej interwencji człowieka. Jego zastosowanie jest szczególnie wartościowe w złożonych i szybko zmieniających się środowiskach chmurowych, gdzie manualne zarządzanie jest nierealne.

3. Metodologia Badawcza

Niniejszy rozdział poświęcony jest szczegółowemu opisowi metodologii badawczej przyjętej w pracy magisterskiej. Celem jest zapewnienie transparentności i powtarzalności przeprowadzonych eksperymentów, a także umożliwienie obiektywnej oceny uzyskanych wyników. Zaprezentowana metodologia stanowi kompleksowe podejście do weryfikacji postawionych hipotez badawczych oraz odpowiedzi na zdefiniowane problemy. Rozdział rozpoczyna się od precyzyjnego sformułowania problemu badawczego i szczegółowych pytań badawczych, które wyznaczają kierunek empirycznej części pracy. Następnie omówione zostaną kluczowe aspekty projektowania eksperymentów, w tym wybór odpowiednich metryk, scenariuszy obciążenia oraz specyfika środowiska testowego. Całość ma na celu stworzenie solidnych ram dla rzetelnej analizy wydajności i efektywności autonomicznych pętli sterowania w środowisku Kubernetes.

3.1. Sformułowanie problemu badawczego i pytań badawczych

Głównym problemem badawczym, do którego rozwiązania dąży niniejsza praca, jest określenie optymalnych strategii autonomicznego zarządzania cyklem życia usług w środowisku Kubernetes, ze szczególnym uwzględnieniem mechanizmów skalowania i alokacji zasobów. W kontekście rosnącej złożoności aplikacji rozproszonych i potrzeby minimalizacji interwencji ludzkiej, kluczowe staje się zrozumienie, w jaki sposób istniejące rozwiązania radzą sobie z dynamicznie zmieniającymi się warunkami obciążenia oraz czy autorskie podejście, oparte na modelu MAPE-K, może zaoferować wyższą efektywność i adaptacyjność. Problem ten dotyczy bezpośrednio wyzwań związanych z utrzymaniem wysokiej dostępności, optymalnego wykorzystania zasobów oraz obniżeniem kosztów operacyjnych w nowoczesnych architekturach chmurowych.

W celu rozwiązania postawionego problemu badawczego, sformułowano następujące szczegółowe pytania badawcze:

1. **Charakterystyka i Ograniczenia Istniejących Mechanizmów:** W jaki sposób VPA, HPA oraz KEDA reagują na różnorodne, dynamiczne wzorce obciążenia (np. narastające obciążenie, obciążenie szczytowe, obciążenie zmienne cyklicznie) w kontekście efektywności alokacji zasobów (CPU, pamięć) i stabilności usług (opóźnienia, błędy)? Jakie są ich fundamentalne ograniczenia w zakresie proaktywnego skalowania i optymalizacji, które wynikają z ich wewnętrznych zasad działania?
2. **Wydajność i Adaptacyjność Rozwiązania MAPE-K:** Czy autorskie rozwiązanie autonomicznej pętli sterującej, zaprojektowane i zaimplementowane w oparciu o model MAPE-K, jest w stanie osiągnąć wyższą efektywność wykorzystania zasobów oraz lepszą responsywność (np. niższe średnie i maksymalne opóźnienia) aplikacji w środowisku Kubernetes w porównaniu do VPA, HPA i KEDA, szczególnie w scenariuszach wymagających zaawansowanej predykcji i koordynacji decyzji?

3. **Wpływ Bazy Wiedzy na Efektywność MAPE-K:** W jakim stopniu komponent bazy wiedzy w architekturze MAPE-K, integrujący historyczne dane o wydajności i predefiniowane polityki adaptacji, przyczynia się do poprawy jakości decyzji podejmowanych przez pętlę sterującą oraz do długoterminowej optymalizacji działania systemu w porównaniu do podejść czysto reaktywnych?
4. **Kryteria Oceny i Metryki Sukcesu:** Jakie kluczowe metryki wydajnościowe (np. średnie zużycie CPU/RAM, procent niewykorzystanych zasobów, czasy odpowiedzi, przepustowość, liczba błędów) oraz wskaźniki operacyjne (np. liczba restartów Podów, stabilność skalowania) należy zastosować, aby obiektywnie porównać skuteczność działania VPA, hpa, KEDA oraz autorskiego rozwiązania MAPE-K w kontrolowanych warunkach eksperymentalnych?

3.2. Środowisko testowe

3.2.1. Warstwa wirtualizacji i sprzętu

W celu przeprowadzenia badań oraz weryfikacji przyjętych założeń projektowych przygotowano dedykowane środowisko testowe. Infrastruktura została zaprojektowana w oparciu o paradygmat Infrastructure as a Code (IaC), co zapewnia powtarzalność procesu wdrażania oraz spójność konfiguracji. Jako platformę bazową wykorzystano środowisko wirtualizacji Proxmox. Na jego potrzeby powołano klaster składający się z trzech maszyn wirtualnych o identycznej specyfikacji sprzętowej. Każdy z węzłów dysponuje następującymi zasobami:

- Procesor (vCPU): 4 rdzenie,
- Pamięć operacyjna (RAM): 8 GB,
- Przestrzeń dyskowa: 100 GB.

Proces powoływania maszyn wirtualnych oraz konfiguracja warstwy sprzętowej zostały zautomatyzowane przy użyciu narzędzia Terraform.

3.2.2. System operacyjny i orkiestracja

Na wszystkich węzłach zainstalowano system operacyjny Ubuntu 24.04 LTS. Stanowi on bazę dla klastra Kubernetes, który został wdrożony przy użyciu lekkiej dystrybucji k3s w wersji v1.33.2+k3s1.

Instalacja oraz wstępna konfiguracja klastra zostały zrealizowane za pomocą narzędzia do zarządzania konfiguracją Ansible, z wykorzystaniem roli k3s-ansible dostępnej w repozytorium ¹. W ramach warstwy sieciowej klastra Container Network Interface (CNI) zastosowano domyślne rozwiązanie dla k3s, czyli Flannel, co zapewnia stabilną komunikację między podami wewnątrz klastra.

Zarządzanie cyklem życia aplikacji oraz konfiguracją klastra odbywa się zgodnie z podejściem GitOps. Kluczowym elementem tej architektury jest narzędzie FluxCD, które stale

¹ <https://github.com/k3s-io/k3s-ansible>

monitoruje repozytorium kodu i automatycznie synchronizuje stan klastra z definicjami zawartymi w systemie kontroli wersji.

Proces dostarczania oprogramowania wspierany jest przez przygotowane potoki Continuous Integration / Continuous Delivery (CI/CD), które odpowiadają za budowanie, testowanie oraz przygotowanie manifestów wdrożeniowych. Takie podejście pozwoliło na pełną automatyzację procesu wdrażania kolejnych zasobów i usług w środowisku badawczym.

Tabela 3.1. Specyfikacja techniczna środowiska eksperymentalnego

Kategoria	Komponent / Specyfikacja
Platforma wirtualizacji	Proxmox
Automatyzacja infrastruktury	Terraform
Konfiguracja systemu	Ansible (k3s-ansible)
Liczba węzłów	3 maszyny wirtualne
Zasoby węzła	4 vCPU, 8 GB RAM, 100 GB HDD
System operacyjny	Ubuntu 24.04 LTS
Orkiestrator	k3s (v1.33.2+k3s1)
Sieć (CNI)	Flannel
Model wdrażania	GitOps (FluxCD)
Narzędzie obciążające	k6 (uruchamiane zewnątrz)
Metodyka testów	Generowanie ruchu HTTP z konsoli lokalnej

3.2.3. Instrumentarium badawcze i narzędzia pomiarowe

W celu realizacji testów wydajnościowych oraz weryfikacji stabilności klastra pod obciążeniem, zastosowano dedykowane narzędzia do generowania ruchu sieciowego.

Jako generator obciążenia wykorzystano narzędzie **k6**, które służy do przeprowadzania testów wydajnościowych. Skrypty testowe k6, przygotowane w języku JavaScript, są uruchamiane lokalnie z konsoli na stanowisku roboczym, a generowane przez nie żądania HTTP kierowane są na zewnątrz klastra Kubernetes, co realistycznie symuluje ruch pochodzący od zewnętrznych użytkowników. Taka metodyka pozwala na mierzenie kluczowych metryk z perspektywy klienta, w tym opóźnień (*latency*), przepustowości (*throughput*) oraz wskaźnika błędów.

Dane obciążeniowe kierowane są na docelową aplikację testową, która została zaprojektowana specjalnie na potrzeby pracy magisterskiej. Architektura i specyfikacja tej aplikacji zostały szczegółowo opisane w kolejnej sekcji dotyczącej eksperymentów.

3.3. Projektowanie eksperymentów

Projektowanie eksperymentów stanowi kluczowy etap niniejszej pracy magisterskiej, mający na celu systematyczną i powtarzalną weryfikację postawionych hipotez oraz udzielenie odpowiedzi na zdefiniowane problemy badawcze. Celem jest obiektywna ocena

wydajności, efektywności oraz zachowania autonomicznych mechanizmów skalowania w środowisku Kubernetes. Eksperymenty zostaną zaprojektowane tak, aby umożliwić bezpośrednie porównanie działania istniejących rozwiązań (VPA, HPA, KEDA) z autorskim podejściem opartym na modelu MAPE-K, w kontrolowanych warunkach obciążenia.

3.3.1. Wybór metryk oceny

Aby rzetelnie ocenić skuteczność poszczególnych mechanizmów autoskalowania, konieczny jest precyzyjny wybór obiektywnych i mierzalnych metryk. W ramach niniejszych badań zostaną uwzględnione następujące kategorie metryk, które pozwolą na wieloaspektową analizę:

1. Metryki efektywności wykorzystania zasobów:

- **Średnie i maksymalne zużycie CPU (%):** Procentowe wykorzystanie procesora przez Pody w stosunku do przydzielonych im zasobów. Wysokie zużycie przy niskim marginesie błędu świadczy o efektywnym wykorzystaniu.
- **Średnie i maksymalne zużycie pamięci (MiB/%):** Analogicznie, pomiar wykorzystania pamięci.
- **Współczynnik marnotrawstwa zasobów (%):** Obliczony jako stosunek niewykorzystanych, ale przydzielonych zasobów do całkowitej ilości przydzielonych zasobów. Niższy współczynnik wskazuje na lepszą optymalizację.
- **Liczba alokowanych vs. faktycznie wykorzystanych zasobów:** Analiza różnic między zasobami żądanymi (requests) i limitowanymi (limits) a rzeczywistym zużyciem.

2. Metryki wydajności usług (Quality of Service - QoS):

- **Średni i maksymalny czas odpowiedzi (Response Time - RT):** Mierzone w milisekundach, od momentu wysłania żądania do otrzymania odpowiedzi. Niższy czas odpowiedzi wskazuje na lepszą responsywność.
- **Przepustowość (Throughput):** Liczba zrealizowanych operacji (np. żądań HTTP) na jednostkę czasu. Wyższa przepustowość oznacza większą zdolność systemu do obsługi obciążenia.
- **Wskaźnik błędów (%):** Procent nieudanych żądań lub operacji w stosunku do całkowitej liczby. Niższy wskaźnik świadczy o stabilności.
- **P95/P99 czasu odpowiedzi:** 95. i 99. percentyl czasu odpowiedzi, wskazujący na doświadczenia większości użytkowników i skalę problemów dla marginalnej grupy.

3. Metryki operacyjne i stabilności:

- **Liczba skalowań (up/down):** Częstotliwość zmian liczby replik lub zasobów, co może świadczyć o stabilności lub "chwiejności" autoskalera.
- **Liczba restartów Podów:** Dotyczy VPA, gdzie zmiana zasobów może wymagać restartu, co wpływa na dostępność.

- **Czas stabilizacji po zmianie obciążenia:** Czas potrzebny systemowi na osiągnięcie stabilnego stanu (np. powrót metryk do pożądanych wartości) po gwałtownej zmianie obciążenia.

Zebrane dane zostaną poddane analizie statystycznej w celu wyciągnięcia wiarygodnych wniosków.

3.3.2. Scenariusze obciążenia i symulacje

Aby zapewnić kompleksową ocenę i porównanie mechanizmów skalowania, eksperymenty zostaną przeprowadzone z wykorzystaniem różnorodnych, realistycznych **scenariuszy obciążenia**. Scenariusze te mają za zadanie odzwierciedlać typowe wzorce ruchu występujące w rzeczywistych środowiskach produkcyjnych, a także testować odporność systemów na nagłe i ekstremalne zmiany. Planuje się zastosowanie następujących typów obciążenia:

- **Obciążenie stałe (Constant Load):** Utrzymywanie stabilnego, umiarkowanego obciążenia przez dłuższy czas, w celu oceny efektywności bazowej alokacji zasobów i stabilności działania autoskalerów.
- **Obciążenie narastające (Ramp-up Load):** Stopniowe zwiększanie obciążenia, aby zbadać zdolność autoskalerów do adaptacji i skalowania w górę w miarę wzrostu zapotrzebowania.
- **Obciążenie szczytowe (Peak Load):** Gwałtowny i wysoki wzrost obciążenia (tzw. "spike"), mający na celu przetestowanie szybkości reakcji i odporności mechanizmów na nagłe, intensywne zapotrzebowanie.
- **Obciążenie zmienne cyklicznie (Cyclic/Diurnal Load):** Symulacja wzorców ruchu charakterystycznych dla cykli dobowych lub tygodniowych (np. niższe obciążenie w nocy, wyższe w ciągu dnia), w celu oceny zdolności autoskalerów do efektywnego skalowania zarówno w górę, jak i w dół.
- **Obciążenie ze zmiennymi wzorcami dostępu (Varying Access Patterns):** Złożone scenariusze, które mogą symulować różne typy operacji (np. odczyty vs. zapisy w bazie danych), aby ocenić, jak mechanizmy radzą sobie z różnorodnym zapotrzebowaniem na zasoby.

Każdy scenariusz zostanie powtórzony wielokrotnie w celu zapewnienia wiarygodności statystycznej wyników.

3.3.3. Narzędzia i środowisko eksperymentalne

Przeprowadzenie rzetelnych eksperymentów wymaga odpowiednio skonfigurowanego środowiska testowego oraz zestawu narzędzi. W niniejszej pracy zostanie wykorzystany następujący zestaw zasobów:

Obciążana usługa testowa

Obciążana usługa testowa została zaimplementowana w języku Python z wykorzysta-

niem frameworka **FastAPI**. Jej głównym zadaniem jest symulowanie zróżnicowanych scenariuszy zużycia zasobów, niezbędnych do weryfikacji funkcjonalności mechanizmów automatycznego skalowania (w tym VPA, HPA, KEDA oraz Operatora MAPE-K). Aplikacja udostępnia trzy dedykowane punkty końcowe (endpoints), z których każdy symuluje inny typ obciążenia:

- `/cpu`: Symuluje obciążenie *CPU-bound* (zależne od procesora) poprzez wykonywanie operacji haszujących (`hashlib.sha256`).
- `/mem`: Symuluje obciążenie *Memory-bound* (związane z pamięcią operacyjną) poprzez dynamiczną alokację dużych tablic (`numpy`).
- `/io`: Symuluje obciążenie *I/O-bound* (wejścia/wyjścia) poprzez operacje zapisu i odczytu plików tymczasowych.

Poprzez zmianę parametrów przekazywanych w żądaniach HTTP do tych endpointów, możliwe jest precyzyjne kontrolowanie intensywności obciążenia klastra. Ponadto, aplikacja posiada wbudowaną instrumentację (`prometheus-fastapi-instrumentator`), która automatycznie wystawia metryki wydajnościowe. Metryki te są zbierane przez system monitorujący klastra, stanowiąc kluczowe dane wejściowe dla wszystkich testowanych metod automatycznego skalowania.

Generator ruchu k6

Jako generator ruchu sieciowego i narzędzie do przeprowadzania testów wydajnościowych wykorzystano **k6**. Jest to narzędzie *open source*, które pozwala na definiowanie skomplikowanych scenariuszy testowych w języku JavaScript.

Kluczową cechą metodologiczną jest fakt, że **skrypty testowe k6 są uruchamiane lokalnie z konsoli** na stanowisku roboczym. Generowane przez nie żądania HTTP kierowane są na zewnętrzny adres klastra Kubernetes (przez serwis typu LoadBalancer), co realistycznie symuluje ruch pochodzący od zewnętrznych użytkowników. Taka konfiguracja umożliwia pomiar kluczowych metryk (np. opóźnienia *latency*) z perspektywy klienta, zapewniając wiarygodność danych wyjściowych testów porównawczych.

Środowisko eksperymentalne (klastr Kubernetes)

Szczegółowa specyfikacja klastra Kubernetes, w tym parametry maszyn wirtualnych, system operacyjny oraz narzędzia do zarządzania infrastrukturą (**Terraform, Ansible, FluxCD**), zostały opisane w podrozdziałach 3.2.1 i 3.2.2. Na potrzeby eksperymentów w tym środowisku wdrożono również kompletny stos monitorujący (**Prometheus, Grafana**), który służy zarówno do zasilania pętli Operatora MAPE-K, jak i jako główne narzędzie do analizy danych i wizualizacji wyników porównawczych wszystkich testowanych mechanizmów skalowania (VPA, HPA, KEDA).

3.3.4. Analiza Wrażliwości i Narzut Systemowy (Overhead)

Weryfikacja efektywności autonomicznej pętli sterującej wymaga nie tylko pomiaru zysków w zakresie wydajności (*Service Level Objectives - SLO*) i optymalizacji kosztów, lecz także krytycznej analizy narzutu systemowego (*overhead*) generowanego przez sam mechanizm monitorowania i sterowania. Agresywne nastawy pętli, w szczególności interwał odpytywania metryk (*polling rate*) ustawiony na 1 s (w przypadku KEDA i Prometheus), stanowią kluczowy czynnik ryzyka, wprowadzający potencjalną niestabilność i przeciążenie.

Narzut systemowy, $\mathcal{O}_{\text{MAPE-K}}$, definiowany jest jako zużycie zasobów klastra, które jest bezpośrednim skutkiem działania pętli sterującej, a nie obciążenia aplikacyjnego. W ramach pracy wyodrębniono trzy główne wektory generowania narzutu przy wysokiej częstotliwości próbkowania:

1. **Saturacja Warstwy Sterowania (*Control Plane Saturation*):** Częste zapytania (GET/LIST zasobów `CustomResourceDefinitions` oraz statusów HPA/Deployment) do `kube-apiserver` i `etcd`. Jest to najbardziej krytyczny punkt obciążenia, prowadzący do dławienia (*throttling*) pozostałych żądań administracyjnych.
2. **Koszty Infrastruktury Monitorującej:** Wzrost zużycia CPU i pamięci RAM przez komponenty takie jak Prometheus (ze względu na konieczność parsowania, indeksowania i zapisu dużej liczby próbek na sekundę) oraz operatorzy skalujący (KEDA Operator).
3. **Narzut Aplikacyjny i Sieciowy:** Konieczność częstego generowania i serializacji metryk wewnątrz aplikacji (`endpoint /metrics`), co w systemach jednowątkowych może prowadzić do wzrostu opóźnień (*latency*) w obsłudze ruchu biznesowego.

Do pomiaru narzutu systemowego wykorzystano zestaw metryk bazujących na systemach monitorujących klastra Kubernetes:

- **Zużycie CPU/RAM kube-apiserver:** Monitorowane za pomocą metryk `kube_pod_container_resource_usage_seconds_total` oraz `usage_seconds_total`.
- **Współczynnik Dławienia (*Throttling Rate*):** Wzrost liczby błędów 429 (`Too Many Requests`) w metryce `apiserver_request_total`.
- **Opóźnienie etcd:** Analiza metryki `etcd_disk_wal_fsync_duration_seconds` jako wskaźnika wydajności persystencji danych klastra.

W celu wyznaczenia opłacalności stosowania agresywnego próbkowania przeprowadzono test A/B, porównując dwie konfiguracje pętli sterującej przy identycznym obciążeniu syntetycznym:

- **Scenariusz A (Baseline):** `Polling Rate = 30 s` (Standardowy).
- **Scenariusz B (Real-time):** `Polling Rate = 1 s` (Agresywny).

Wyniki testu umożliwią określenie punktu, w którym narzut $\mathcal{O}_{\text{MAPE-K}}$ staje się istotny, a dalsze zwiększanie częstotliwości próbkowania przestaje przynosić korzyści w stabilizacji wydajności aplikacji.

3.4. Metody analizy danych

Po zakończeniu fazy gromadzenia danych z przeprowadzonych eksperymentów, kluczowe jest zastosowanie odpowiednich metod analizy, które umożliwią wyciągnięcie wiarygodnych wniosków i weryfikację postawionych hipotez badawczych. Proces analizy danych będzie obejmował zarówno podejścia ilościowe, jak i jakościowe, a także wykorzystanie specjalistycznych narzędzi do wizualizacji i interpretacji wyników. Celem jest nie tylko stwierdzenie różnic w wydajności poszczególnych mechanizmów skalowania, ale również zrozumienie przyczyn obserwowanych zachowań.

3.4.1. Analiza ilościowa i jakościowa

Analiza danych zostanie przeprowadzona dwutorowo, łącząc metody ilościowe i jakościowe w celu uzyskania kompleksowego obrazu funkcjonowania badanych mechanizmów:

1. **Analiza Ilościowa:** Będzie stanowiła podstawę weryfikacji hipotez i opierać się na statystycznym przetwarzaniu zebranych metryk numerycznych. Kluczowe aspekty analizy ilościowej obejmują:
 - **Statystyka opisowa:** Obliczenie średnich, median, odchyłeń standardowych, minimum i maksimum dla wszystkich zebranych metryk (np. czasu odpowiedzi, zużycia CPU, przepustowości). Pozwoli to na wstępne scharakteryzowanie danych i identyfikację rozkładów.
 - **Analiza trendów:** Badanie zmian metryk w czasie dla różnych scenariuszy obciążenia, co pozwoli na ocenę dynamiki reakcji każdego autoskalera.
 - **Analiza korelacji:** Zbadanie związków między różnymi metrykami (np. między obciążeniem a czasem odpowiedzi, lub między liczbą Podów a zużyciem zasobów).
 - **Testy porównawcze:** Zastosowanie odpowiednich testów statystycznych (np. testów t-Studenta, analizy wariancji ANOVA) do porównania średnich wartości metryk między poszczególnymi mechanizmami skalowania (VPA, HPA, KEDA, MAPE-K) w celu określenia istotności statystycznej obserwowanych różnic.
 - **Analiza percentylowa:** Obliczenie percentyli (np. P95, P99) dla metryk wydajności, co jest kluczowe dla oceny doświadczeń użytkowników i identyfikacji ewentualnych anomalii lub "ogonów" rozkładu.
2. **Analiza Jakościowa:** Uzupełni analizę ilościową, dostarczając głębszego zrozumienia przyczyn obserwowanych zachowań, zwłaszcza w przypadku anomalii lub niespodziewanych wyników. Obejmie ona:
 - **Analiza logów systemowych i zdarzeń Kubernetesa:** Przeglądanie logów z komponentów autoskalerów, kubeleta i kontrolera Kubernetes w celu zidentyfikowania konkretnych decyzji, błędów lub zdarzeń, które mogły wpłynąć na zachowanie systemu.

- **Analiza konfiguracji i polityk:** Szczegółowa ocena, jak specyficzne konfiguracje i algorytmy każdego autoskalera wpływały na jego reakcje w danych scenariuszach.
- **Studia przypadków (Case Studies):** Wybór konkretnych, interesujących momentów w przebiegu eksperymentów (np. nagłe skoki obciążenia, awarie, momenty stabilizacji) i ich dogłębna analiza jakościowa w celu wyjaśnienia obserwowanych zjawisk.

Połączenie obu podejść pozwoli na kompleksową interpretację wyników, łącząc precyzję danych numerycznych z kontekstualnym zrozumieniem mechanizmów działania.

3.4.2. Narzędzia do wizualizacji i interpretacji wyników

Efektywna wizualizacja danych jest niezbędna do szybkiego zrozumienia złożonych zbiorów danych i komunikowania wyników badań. Do wizualizacji i interpretacji wyników eksperymentów zostaną wykorzystane następujące narzędzia:

- **Grafana:** To narzędzie typu open-source do tworzenia interaktywnych dashboardów. Zostanie użyta do wizualizacji metryk czasowych (np. zużycie CPU/RAM w funkcji czasu, liczba replik, czasy odpowiedzi) zbieranych z Prometheus. Pozwoli to na dynamiczne śledzenie zmian i porównywanie zachowań różnych autoskalerów.
- **Jupyter Notebooks (z bibliotekami Python, np. Pandas, Matplotlib, Seaborn):** Środowisko to zostanie wykorzystane do zaawansowanej analizy statystycznej i generowania niestandardowych wykresów. Umożliwi ono szczegółowe przetwarzanie danych, przeprowadzanie testów statystycznych oraz tworzenie wysokiej jakości wizualizacji (np. wykresów słupkowych, liniowych, pudełkowych, rozrzutu), które zostaną włączone do pracy dyplomowej.
- **Arkusze kalkulacyjne (np. Microsoft Excel, Google Sheets):** Do wstępnej analizy danych, ich agregacji oraz prostych obliczeń statystycznych. Może być również wykorzystany do organizacji surowych danych.
- **Narzędzia do analizy logów (np. Loki, ELK Stack):** Służą do przeszukiwania, agregacji i analizy logów systemowych, co jest kluczowe dla jakościowej oceny zachowania autoskalerów i wykrywania problemów.

Wykorzystanie tych narzędzi zapewni zarówno możliwość dogłębnej analizy statystycznej, jak i czytelną prezentację uzyskanych rezultatów, co jest kluczowe dla skutecznej komunikacji wniosków badawczych.

4. Implementacja i Badanie Mechanizmów Skalowania w Kubernetesie

4.1. Opis Środowiska Testowego

4.2. Konfiguracja i Wdrożenie Mechanizmów Skalowania

4.2.1. Konfiguracja i Wdrożenie VPA (Vertical Pod Autoscaler)

4.2.2. Konfiguracja i Wdrożenie HPA (Horizontal Pod Autoscaler)

4.2.3. Konfiguracja i Wdrożenie KEDA (Kubernetes Event-driven Autoscaling)

4.3. Wyniki Eksperymentów i Analiza

4.3.1. Wyniki Eksperymentów z VPA i Analiza

4.3.2. Wyniki Eksperymentów z HPA i Analiza

4.3.3. Wyniki Eksperymentów z KEDA i Analiza

4.4. Porównanie i Synteza Wyników (VPA, HPA, KEDA)

4.4.1. Analiza Porównawcza Wydajności, Efektywności i Zastosowań

5. Projekt i Implementacja Własnego Rozwiązania Opartego na MAPE-K

5.1. Założenia Projektowe i Architektura Rozwiązania

5.1.1. Definicja poszczególnych komponentów (Monitor, Analize, Plan, Execute, Knowledge Base)

5.1.2. Wykorzystane technologie i biblioteki

5.2. Implementacja Komponentów

5.2.1. Moduł Monitorowania (pozyskiwanie metryk)

5.2.2. Moduł Analizy (algorytmy decyzyjne)

5.2.3. Moduł Planowania (generowanie akcji)

5.2.4. Moduł Wykonawczy (interakcja z API Kubernetesa)

5.2.5. Baza Wiedzy (przechowywanie polityk, historii)

5.3. Wyniki Eksperymentów z Własnym Rozwiązaniem

5.3.1. Scenariusze testowe i obciążenia

5.3.2. Analiza wydajności i efektywności w porównaniu do istniejących mechanizmów

5.3.3. Dyskusja nad zaletami i wadami podejścia

6. Analiza Wyników i Dyskusja

6.1. Interpretacja uzyskanych wyników w kontekście celów pracy

6.2. Weryfikacja hipotez badawczych

6.3. Wnioski z badań i ich implikacje praktyczne

6.4. Ograniczenia przeprowadzonych badań

7. Podsumowanie i Dalsze Kierunki Badań

7.1. Podsumowanie kluczowych osiągnięć pracy

7.2. Propozycje dalszych badań i rozwoju rozwiązania

Spis rysunków

Spis tabel

Spis załączników

1. Nazwa załącznika 1	35
2. Nazwa załącznika 2	36

Załącznik 1. Nazwa załącznika 1

Załącznik 2. Nazwa załącznika 2